WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT BASED LEARNING — PROJECT 69

PRINTING PRESS JOB ORDER AND PRODUCTION MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted towards the Database Management Systems PBL evaluation

SUBMITTED BY

Om Pilaji | 25WU0102202

B.Tech — Computer Science and Engineering

Academic Year 2026–2027

Faculty Guide: _____

Submission Date: _____

REVIEW SHEET

Review	Week	Marks	Expected evidence
1	7	5	Problem statement, ER diagram, relational schema
2	11	5	3NF, DDL, data, SQL queries, prototype
3	16	5	Python app, demonstration, source, report, viva

Faculty evaluation and signatures to be completed by the department.

DECLARATION

I, Om Pilaji (25WU0102202), submit this report for Database Management Systems Project Based Learning. It documents the PrintFlow reference application and database design included with this submission. The execution and verification details stated in the report distinguish tested backend behavior from features that require further work.

Student Signature: _____ Date: _____

ACKNOWLEDGEMENT

I thank the faculty of the School of Technology, Woxsen University, for providing the opportunity to apply database concepts to a real business workflow. The project helped me practice entity modelling, relational integrity, application development and transaction validation.

ABSTRACT

PrintFlow is an educational printing press management application that organizes customer information, quotations, job orders, paper and ink inventory, machine reservations, production progress, deliveries, invoices and payments. The backend uses Python FastAPI and SQLAlchemy with an SQLite demonstration mode and PostgreSQL support. The user interface is a responsive React/Vite dashboard. Validation logic protects against negative stock, overlapping machine schedules in sequential requests, delivery quantities beyond orders and payment amounts exceeding invoice totals. This report covers the official three-review deliverables and provides implementation details, testing evidence and limits of the prototype.

CONTENTS

1 Introduction

2 Problem Statement and Objectives

3 Requirements and Architecture

4 Database Design and E-R Representation

5 Normalization and Integrity Constraints

6 Implementation and User Workflows

7 SQL Queries and Reports

8 Testing and Results

9 Limitations and Future Scope

10 Conclusion

References

Appendix - Run instructions

CHAPTER 1 — INTRODUCTION

Printing presses handle diverse customer orders such as booklets, pamphlets, brochures and event materials. Managing quotation, stock, production stages, machine assignments and billing in disconnected records increases the risk of duplicate data and errors. A relational database supports traceable and consistent workflows.

CHAPTER 2 — PROBLEM STATEMENT AND OBJECTIVES

Problem: Paper-based job cards make it difficult to monitor print jobs, track materials, avoid conflicting machine schedules, verify deliveries and reconcile payments.

- Centralize customer, quotation and job information
- Control materials through stock issue transactions
- Prevent simple overlapping machine allocations
- Track ordered and delivered quantities
- Calculate invoice balances from payments
- Provide a working Python application with SQL reporting

SCOPE

The current prototype supports a local demonstration of key printing press processes. It does not include live payments, procurement integration or enterprise-scale multi-user concurrency.

CHAPTER 3 — REQUIREMENTS AND ARCHITECTURE

Layer	Implementation
User interface	React 18 + Vite + responsive CSS
Application API	FastAPI + Pydantic
Persistence	SQLAlchemy ORM
Database	SQLite demo; PostgreSQL supported with DATABASE_URL
Reporting	API dashboard summaries + SQL report statements

High-level data flow: Browser -> FastAPI routes -> transaction validation -> SQLAlchemy session -> relational database -> JSON reports.

FUNCTIONAL REQUIREMENTS

- Create and manage customer records, materials and machines
- Create quotations and associate them with jobs
- Reserve production windows without detectable scheduling overlaps
- Issue material while enforcing available stock
- Capture production stages and ordered delivery quantities
- Record invoices and valid payments, view revenue and dues
- Search master and transactional records using the application interface

CHAPTER 4 — DATABASE DESIGN

The application defines 11 interconnected tables: customers, materials, machines, quotations, jobs, material_issues, schedules, stages, deliveries, invoices and payments. Core identifiers use surrogate integer primary keys. Job numbers and machine/material names are unique.

Entity	Key relations
QUOTATIONS	customer_id -> CUSTOMERS
JOBS	customer_id -> CUSTOMERS; quotation_id -> QUOTATIONS
MATERIAL_ISSUES	job_id -> JOBS; material_id -> MATERIALS
SCHEDULES	job_id -> JOBS; machine_id -> MACHINES
STAGES	job_id -> JOBS
DELIVERIES	job_id -> JOBS
INVOICES	job_id -> JOBS (unique)
PAYMENTS	invoice_id -> INVOICES

SIMPLIFIED E-R RELATIONSHIP MAP

CUSTOMERS (1) -> (M) QUOTATIONS

CUSTOMERS (1) -> (M) JOBS <- (0..1) QUOTATIONS

JOBS (1) -> (M) MATERIAL_ISSUES (M) -> (1) MATERIALS

JOBS (1) -> (M) SCHEDULES (M) -> (1) MACHINES

JOBS (1) -> (M) STAGES and DELIVERIES

JOBS (1) -> (0..1) INVOICES (1) -> (M) PAYMENTS

CHAPTER 5 — NORMALIZATION AND CONSTRAINTS

Tables are separated by independent entity, and repeating groups such as material issues, stage logs and payments are represented as separate relations. Each fact is primarily dependent on its table key. This design follows the central ideas of 1NF, 2NF and 3NF. A formal proof for all possible business functional dependencies would require a complete enterprise data dictionary.

Control	Enforcement
Unique job number	Database UNIQUE
Positive order quantity	Database CHECK
Stock cannot become negative	Transaction check plus stock CHECK
No double machine booking	Application overlap check (not concurrent-safe)
Ordered delivery cap	Application validation plus DB CHECK on job balance
Invoice payment cap	Application aggregation check

Production stage sequence	Application-level predecessor validation

For high-concurrency PostgreSQL deployment, machine overlap should additionally be enforced by an EXCLUDE constraint with a timestamp range.

CHAPTER 6 — APPLICATION IMPLEMENTATION

The FastAPI server exposes documented /api endpoints with a generic resource pattern for master and transactional creation, and /api/reports/summary for dashboard metrics. The React frontend offers individual registers, form entry, client-side search and a summary dashboard.

Module	Action
Customers	Create, list, edit, delete where unused
Quotations	Draft quotation, quantity and unit price
Jobs	Unique job number, customer and quotation association
Materials	Stock and threshold, issue per job
Machines	Create machine and add time-window bookings
Stages	Record DESIGN -> PREPRESS -> PRINTING -> FINISHING -> PACKING
Deliveries	Partial/full deliveries with cap
Billing	One invoice per job and payments within outstanding total

CHAPTER 7 — SQL QUERIES AND REPORTS

The source package includes database/reports.sql with SELECT queries for pending jobs, total material usage, machine schedules, job stage status, outstanding delivery, revenue, invoice balances, low-stock materials and estimated material cost. The dashboard computes summary values using SQL aggregate functions and displays the resulting records.

CHAPTER 8 — TESTING AND RESULTS

A local FastAPI TestClient smoke test was executed against a temporary SQLite database. Fifteen assertions passed, including creation of core records, rejection of insufficient stock, overlapping machine reservations, excess delivery and overpayment, as well as health, listing and revenue checks.

Test category	Observed result
Backend HTTP health	Passed
Create customer, material, quotation, job	Passed
Material issue and overspend rejection	Passed
Machine overlap rejection	Passed
Ordered stage progression	Passed

Delivery limit rejection	Passed
Invoice and overpayment rejection	Passed
Dashboard revenue query	Passed

Test evidence is generated by tests/smoke_test.py. These test outcomes cover the SQLite backend only. The React interface and a connected PostgreSQL deployment were not independently verified in this build. Live application screenshots are therefore not represented as verified evidence.

CHAPTER 9 — LIMITATIONS AND FUTURE SCOPE

- Introduce authentication, role-based access and robust authorization
- Add schema migrations, backups and audit trail
- Use PostgreSQL EXCLUDE to prevent overlapping machine bookings under concurrency
- Add material reservation, procurement, suppliers and stock replenishment
- Support production stage updates and richer order details
- Add invoice numbering, printable bills, PDF export and real production screenshots
- Implement end-to-end browser tests and production deployment

CHAPTER 10 — CONCLUSION

PrintFlow demonstrates how relational design and a Python application can manage printing press business records in a single workflow. Its core value is consistent order, stock, scheduling, delivery and payment information. The included reference implementation and report satisfy the principal structural learning objectives of the selected DBMS PBL topic, while noting specific areas that need further verification or extension.

REFERENCES

1. Woxsen University - DBMS: 70 Unique Project-Based Learning Statements, Project 69 (provided assignment document).

2. PostgreSQL Documentation - https://www.postgresql.org/docs/

3. SQLAlchemy Documentation - https://docs.sqlalchemy.org/

4. FastAPI Documentation - https://fastapi.tiangolo.com/

5. React Documentation - https://react.dev/

APPENDIX — LOCAL EXECUTION

Backend: create a Python environment in backend/, install requirements.txt and run uvicorn main:app --reload --port 8000. Frontend: run npm install and npm run dev in frontend/, then browse http://localhost:5173. To use PostgreSQL, first create printing_press_db and supply a valid DATABASE_URL. See README.md for complete instructions.